

TECHNICAL DEFENSE & VIVA EXAMINATION GUIDE

Sunrise School ERP — 100 Database & Backend Viva Questions

Comprehensive Project-Specific Verification & Defense Guide for PostgreSQL 18 & FastAPI Backend

🔒 100 Project-Specific Questions

📊 61 Active Populated Tables

💰 28 In-Depth Fee & Ledger Questions

⚡ Data Flows, ER Rules & Constraints

★ Top 20 High-Yield Exam Questions

MODULE 1: ARCHITECTURE, MULTI-TENANCY & CORE DESIGN RULES

Q1 What multi-tenancy architecture does Sunrise ERP use, and how is it enforced at the database level?

Sunrise ERP uses a **shared-database, shared-schema multi-tenant architecture with column-level discriminator (`school_id`)**.

- Every school is an independent tenant (customer), not a branch of a single institution.
- Almost every model inherits from `TenantBase` (declared in `backend/app/models/base.py`), which automatically injects `school_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("schools.id", ondelete="CASCADE"), nullable=False, index=True)`.
- Compound unique constraints throughout the schema explicitly prefix with `school_id` (e.g., `UniqueConstraint("school_id", "admission_no")` on `students`, `UniqueConstraint("school_id", "invoice_no")` on `fee_invoices`).
- Only three operational tables in the system do not carry `school_id`:
 1. `schools` (which represents the tenant itself),
 2. `permissions` (global software authorization vocabulary belonging to the codebase, not custom to any school),
 3. `scheduled_jobs` (global system worker registry).

Q2 Why does the project separate `students` from `enrolments`, and what bug does this prevent?

The split separates **lifetime facts** from **session/year-scoped facts**:

- **students** stores facts that belong to the child for their entire schooling lifetime: `id`, `admission_no`, `full_name`, `dob`, `gender`, `blood_group`, `status`.
- **enrolments** stores facts that belong to a single academic year: `enrolment_id`, `student_id`, `class_section_id`, `academic_year_id`, `roll_no`, `status`.
- **Why this exists:** Academic marks, attendance records, fee invoices, bus seat assignments, and class teacher allocations point to `enrolment_id`, **never** directly to `student_id`.
- **Bug prevented:** If tables pointed to `student_id`, promoting a student from Class 9 to Class 10 at the start of a new academic year would silently overwrite or mix Class 9 attendance, fees, and marks with Class 10. By keying to `enrolment_id`, every session's records are completely isolated and historical reports remain intact.

Q3 What is the financial immutability rule implemented in the database, and how does it work?

Financial money rows are **NEVER** updated or deleted in place.

1. **Invoices:** Once created in `fee_invoices`, an invoice amount is never edited. If billed incorrectly, the invoice `status` is changed to `'void'`, an audit reason is captured in `void_reason`, and a brand-new invoice is issued.
2. **Payments:** If an administrative error or bounced cheque occurs, the original `fee_payments` row is never deleted or altered. Instead, a **contra payment entry** is inserted into `fee_payments`:
 - It carries a **negative amount** (`-amount`),
 - It sets `reverses_payment_id = original_payment.id`,
 - It records a mandatory audit `reason`,
 - Corresponding negative rows are inserted into `payment_allocations` to un-settle the invoice lines.
 - This guarantees that accounting ledgers can be audited chronologically without loss of financial history.

Q4 Why is there no `balance\_due` or `outstanding\_fees` column stored on the `students` or `enrolments` tables?

A balance is **always a runtime SUM()**, never a stored column.

- Storing a pre-computed balance introduces cache invalidation bugs (e.g., if a payment is reversed, a concession applied, or an invoice line voided, a denormalized balance column easily goes out of sync).
- In Sunrise ERP, a child's outstanding fees are computed directly by SQL aggregating:

Formula: $\text{Outstanding} = \sum(\text{fee_invoice_lines.amount}) - \sum(\text{fee_invoice_lines.discount}) - \sum(\text{payment_allocations.amount})$

- When an administrator queries `/admin/fees/defaulters` or `/admin/students`, the database calculates the live net balance dynamically using grouped joins.

Q5 How does the system handle destructive database actions, and which table tracks them?

All destructive actions (voiding an invoice, reversing a fee payment, deleting a notice, deactivating a user, or changing academic year status) require an explicit user-typed justification.

- The backend enforces this via the `AuditAction` service (`backend/app/services/audit.py`).
- Every destructive event is written to the `audit_log` table with:
 - `school_id`,
 - `user_id` (the staff member executing the action),
 - `action` (e.g. `'fee.invoice.void'`, `'fee.payment.reverse'`, `'comms.notice.delete'`),
 - `entity_type` and `entity_id` (the affected target table and record ID),
 - `reason` (the exact string entered by the user in the confirmation dialog),
 - `before_state` and `after_state` (JSON snapshots of the modified row),
 - `ip_address` and `created_at`.

Q6 How are sequential human-readable numbers (like Admission No, Invoice No, Receipt No) generated safely without race conditions?

They are managed by the `number_sequences` table using database row-level locking.

- Columns: `school_id`, `entity_type` (`'admission'`, `'invoice'`, `'receipt'`), `prefix` (e.g. `'INV-'`), `current_val`, `padding` (e.g. `6`).
- When a new number is generated, the backend executes `SELECT current_val FROM number_sequences WHERE school_id = :sid AND entity_type = :type FOR UPDATE`.
- `FOR UPDATE` places an exclusive lock on that specific sequence row until the transaction commits.
- The backend increments `current_val += 1`, formats the string (e.g. `INV-2025-000142`), updates the row, and returns it. This guarantees no duplicate numbers even under concurrent billing requests.

Q7 How does the `settings` table work, and why does an unset setting NOT fail?

`settings` stores typed key-value pairs (`key`, `value` JSON, `school_id`).

- It follows **Level 1 and Level 3 configuration** (`backend/app/core/settings_registry.py`).
- The application defines hardcoded registry defaults in Python (`DEFAULTS` dictionary).
- When `school_settings.get(db, school_id, key)` is called:
 - It checks if a row exists in `settings` for that `school_id` and `key`.
 - If found, it returns the custom `Setting.value`.
 - If `None` (unset), it returns the default defined in the Python registry without writing to the database.
- This prevents database bloat and ensures newly introduced configuration flags work across all tenant schools without running complex data migrations.

Q8 What tables power Role-Based Access Control (RBAC), and what is the relationship between them?

Five tables form the security foundation:

1. **users**: Authentication credentials (email , hashed_password , role).
2. **roles**: System roles (Admin , Principal , Teacher , Accountant) and custom roles created by schools.
3. **permissions**: 146 granular capability codes (e.g., fees.invoice.read , students.profile.write , exam.marks.lock).
4. **role_permissions**: Many-to-Many junction table linking role_id to permission_code .
5. **user_roles**: Many-to-Many junction table linking user_id to role_id .
 - The database enforces referential integrity so deleting a role removes its associated role_permissions and user_roles entries automatically via foreign key cascading.

Q9 What data types are standard for Currency, Dates, and IDs across the schema?

- **Currency & Amounts**: Strictly numeric(10, 2) or numeric(12, 2) (e.g., on fee_invoices , fee_payments , salary_structures). FLOAT is strictly prohibited to prevent floating-point rounding errors.
- **Timestamps**: timestamptz (timestamp with time zone) throughout, storing UTC on the wire and rendered to Lucknow office local time (Asia/Kolkata) at the presentation layer.
- **Dates**: date without time zone for calendar events (dob , issued_on , due_date , attendance.date).
- **Primary Keys**: bigint (auto-incrementing 64-bit integer sequences via PostgreSQL BIGSERIAL / IDENTITY).

Q10 How does `academic\_years` handle overlapping sessions (e.g., in January when next year's admissions start while the current year is still teaching)?

In academic_years , a session has lifecycle statuses: draft , active , closing , closed .

- Columns: id , school_id , code (e.g. '2025-26') , start_date , end_date , status , is_current .
- The schema does **not** restrict the database to only one open academic year.
- Multiple academic years can exist simultaneously in the database with status = 'active' . For example, 2025-26 remains active for classes, attendance, and quarterly fees, while 2026-27 is active in admission_cycles to accept entrance applications and interview evaluations.

Q11 How are custom user-defined fields stored without modifying the PostgreSQL table schemas?

Via the custom_fields table.

- Columns: id , school_id , entity_type ('student' , 'employee' , 'enquiry'), field_name , field_label , field_type ('text' , 'number' , 'date' , 'dropdown'), options (JSON array for dropdowns), is_required .
- Dynamic values entered by users are stored inside the custom_data JSONB column on the corresponding entity row (e.g., on students.custom_data or employees.custom_data).
- This allows schools to add attributes like "Aadhaar Card No" or "Bus Route Stop Preference" without executing ALTER TABLE DDL migrations.

Q12 What is `TenantBase` vs `TimestampedBase` in `backend/app/models/base.py`?

- **TimestampedBase** provides the standard primary key id: BigInteger and audit timestamps: created_at (default now()) and updated_at (updated on modification).
- **TenantBase** inherits from TimestampedBase and adds the non-nullable foreign key school_id: ForeignKey("schools.id", ondelete="CASCADE") .
- Any table that holds data belonging to a school inherits from TenantBase . Tables that represent global system concepts inherit directly from TimestampedBase .

MODULE 2: STUDENTS, ENROLMENTS & GUARDIANS

Q13 What is the exact relationship between `students`, `guardians`, and `student\_guardian`?

It is a **Many-to-Many relationship** modeled via the junction table `student_guardian`:

- A student can have multiple guardians (Father, Mother, Local Guardian).
- A guardian can have multiple children enrolled in the same school (Siblings).
- **student_guardian junction columns:**
 - `student_id` (FK `students.id`)
 - `guardian_id` (FK `guardians.id`)
 - `relation` (Enum: `father`, `mother`, `local_guardian`, `other`)
 - `is_primary` (Boolean: indicates who receives official SMS/fee alerts)
 - `is_emergency_contact` (Boolean)
 - `pickup_authorized` (Boolean: verified at the school gate)
- **Unique constraint:** `UniqueConstraint("student_id", "guardian_id")` prevents linking the same guardian twice to the same student.

Q14 How does the system enforce sibling discounts at the database level using `guardians`?

When an enquiry or application is checked for sibling concession:

1. The query joins `student_guardian` on `guardian_id` where `students.status = 'active'`.
2. If another student row shares the same `guardian_id` (specifically where `relation IN ('father', 'mother')`), the backend identifies an active sibling relationship.
3. The system checks `settings.key = 'fees.sibling_concession_percent'` (default `10%`) and automatically proposes a `fee_concessions` record for the younger sibling.

Q15 What columns exist on `enrolments`, and how does it link a student to a physical classroom?

`enrolments` bridges `students` to `class_sections`:

- **Columns:** `id`, `school_id`, `student_id` (FK), `class_section_id` (FK), `academic_year_id` (FK), `roll_no` (Integer), `enrolled_on` (Date), `status` (Enum: `active`, `promoted`, `transferred`, `withdrawn`), `exit_date`, `exit_reason`.
- **Constraint:** `UniqueConstraint("school_id", "academic_year_id", "student_id")` ensures a student can hold only **one** active enrolment per academic year.
- **Constraint:** `UniqueConstraint("school_id", "class_section_id", "academic_year_id", "roll_no")` prevents two students in the same class section from holding the same roll number.

Q16 What happens in the database if a student leaves the school (transfers/withdraws)?

1. The student is **never deleted** (`DELETE FROM students` is prohibited to preserve historical CBSE records and audit compliance).
2. The `enrolments.status` is updated to `'withdrawn'` or `'transferred'`.
3. `enrolments.exit_date` and `enrolments.exit_reason` are populated.
4. `students.status` is set to `'inactive'`.
5. An entry is written to `audit_log` with the transfer certificate reason.
6. The child automatically stops appearing on active daily attendance registers and future monthly fee invoice generation runs.

Q17 What indexes are placed on the `students` table to ensure fast search in the office directory?

The `students` table has indexes on:

1. `(school_id, admission_no)` — Unique B-tree index for instant lookup when typing admission numbers.
2. `(school_id, status)` — Index for filtering active rosters vs alumni.
3. `full_name` — Trigram or B-tree prefix index for rapid case-insensitive office name search (`LIKE 'Sharma%'`).
4. `school_id` — Multi-tenant partition index injected by `TenantBase`.

Q18 What is the foreign key cascade behavior between `schools` and `students`?

`onDelete="CASCADE" :`

- If a tenant `School` row is deleted, all child records (`students` , `enrolments` , `fee_invoices` , etc.) are cascaded and deleted by PostgreSQL.
- However, between `students` and `enrolments` , the constraint is `RESTRICT` / default `NO ACTION` on critical transactional tables: you cannot delete a student row if an `enrolment` or `fee_payment` references it.

Q19 Where is a student's medical and blood group information stored?

Directly in the `students` table:

- Columns: `blood_group` (`varchar(5)`), `medical_notes` (`text`), `emergency_contact_name` (`varchar(120)`), `emergency_contact_phone` (`varchar(20)`).
- This is stored on `students` because a child's blood group and allergies are lifetime physiological facts that do not change from year to year.

Q20 Can a student exist without an enrolment in the database?

Yes.

- When a student is first admitted or their biodata is drafted, a row is created in `students` .
- Until they are assigned to a specific class section (e.g. Class 1-A) for the active academic year, no row exists in `enrolments` .
- Furthermore, an alumnus who graduated retains their `students` row indefinitely, but has no `enrolments` row for current or future academic years.

Q21 What table tracks student house allocations or second languages?

`students` has optional columns `house` (`varchar(30)`) and `second_language` (`varchar(30)`). Additional institutional attributes are stored dynamically inside `students.custom_data` (JSONB) mapped to definitions in `custom_fields` .

Q22 What happens if two students have the same roll number in the same section?

The database rejects the `INSERT` or `UPDATE` with a PostgreSQL integrity violation:

```
psycopg.errors.UniqueViolation: duplicate key value violates unique constraint "uq_enrolments_class_roll".
```

This is enforced by `UniqueConstraint("class_section_id", "academic_year_id", "roll_no")` .

MODULE 3: DEEP-DIVE ON FEES, INVOICING, PAYMENTS & DEFAULTERS**Q23 What are the 8 active tables involved in the Fees module, and what is the specific role of each?**

1. **fee_heads** : The master catalog of named billing components (e.g., Tuition Fee, Development Charge, Transport Fee, Examination Fee).
2. **fee_plans** : Represents the annual fee structure assigned to a class grade (e.g. "Class 10 Fee Plan 2025-26").
3. **fee_plan_items** : The individual line items of a fee plan, defining which `fee_head` is charged, the monetary `amount` , and the billing frequency (monthly, quarterly, annual).
4. **fee_concessions** : Approved student discounts (scholarships, sibling concessions, staff child discounts) with percentage/amount and audit approval.
5. **fee_invoices** : The header bill issued to a student enrolment for a specific calendar month and academic year.
6. **fee_invoice_lines** : The itemized line charges of an invoice (Head, Base Amount, Discount Amount).
7. **fee_payments** : Represents physical money received at the counter or bank (Receipt No, Amount, Payment Method, Idempotency Key).
8. **payment_allocations** : The atomic ledger junction mapping exactly how many rupees of a `fee_payment` settled which specific `fee_invoice_line` .

Q24 What is the exact data flow and database operation when the admin generates monthly fee invoices for a class?

When `POST /admin/fees/generate` is triggered:

- 1. Enrolment Fetch:** The backend queries `enrolments` WHERE `class_section_id = :id` AND `status = 'active'` AND `academic_year_id = :year_id`.
- 2. Plan Lookup:** It identifies the active `fee_plans` and its `fee_plan_items` for that class.
- 3. Existing Check:** It queries `fee_invoices` to ensure no invoice already exists for each `(enrolment_id, period_month, period_year)` with `status ≠ 'void'`.
- 4. Concession Lookup:** It queries active rows in `fee_concessions` WHERE `enrolment_id = :enrolment_id` AND `status = 'approved'`.
- 5. Invoice Insertion:** Inside a database transaction:
 - Inserts a row into `fee_invoices`: `invoice_no` (via `number_sequences`), `enrolment_id`, `academic_year_id`, `period_month`, `period_year`, `issued_on`, `due_date`, `status = 'issued'`.
- 6. Invoice Lines Insertion:** For every applicable `fee_plan_item`:
 - Calculates base `amount`.
 - Computes `discount` if an approved concession matches the `fee_head_id`.
 - Inserts a row into `fee_invoice_lines`: `invoice_id`, `fee_head_id`, `description`, `amount`, `discount`.
- 7.** The transaction commits. No payment or allocation rows are created at this time.

Q25 What happens in the database when a student pays fees at the counter?

When `POST /admin/fees/payments` is submitted:

- 1. Validation:** Checks that the paying `enrolment_id` exists and payment `amount > 0`.
- 2. Idempotency Check:** Verifies `idempotency_key` does not already exist in `fee_payments` for this `school_id`.
- 3. Insert Payment:** Inserts into `fee_payments`:
 - `id`, `school_id`, `enrolment_id`,
 - `receipt_no` (retrieved from `number_sequences`),
 - `amount`, `method` (`cash`, `cheque`, `upi`, `neft`),
 - `instrument_ref` (e.g. Cheque/Transaction UTR No),
 - `received_at = now()`, `received_by = user.id`, `status = 'cleared'`.
- 4. FIFO Allocation Algorithm:**
 - Queries all unsettled `fee_invoice_lines` for this enrolment, ordered by `fee_invoices.due_date ASC, fee_invoice_lines.id ASC`.
 - For each line, calculates remaining balance: $\text{Line Due} = (\text{amount} - \text{discount}) - \sum(\text{existing allocations})$.
 - Allocates the payment amount to lines oldest-first.
 - For each covered line, inserts a row into `payment_allocations`:
`payment_id = payment.id`, `invoice_line_id = line.id`, `amount = allocated_amount`.
- 5. Invoice Status Update:**
 - For each affected invoice, checks if all child lines are fully settled.
 - If sum of allocations equals sum of lines net amounts: updates `fee_invoices.status = 'paid'`, `settled_on = today`.
 - If partially settled: updates `fee_invoices.status = 'partially_paid'`.
- 6.** Transaction commits atomically.

Q26 Explain the structure and purpose of the `payment\_allocations` table.

`payment_allocations` is the **single source of truth for fee realization**.

- **Columns:** `id`, `school_id`, `payment_id` (FK `fee_payments.id`), `invoice_line_id` (FK `fee_invoice_lines.id`), `amount` (`numeric(10,2)`), `created_at`, `updated_at`.
- **Why it points to `fee_invoice_lines` instead of `fee_invoices`:**
 - A school needs to know exactly which head was collected: did the family pay the Tuition Fee or the Transport Fee?
 - Accounting ledgers and tax reports require head-wise revenue recognition.
 - By allocating at the line level, partial payments are accurately split across specific heads.

Q27 How does the database calculate the fee defaulters list? Write the conceptual SQL logic.

The backend (`backend/app/services/fees.py::defaulters`) runs an aggregation query joining invoices, lines, and allocations:

```
SELECT
    e.id AS enrolment_id,
    s.id AS student_id,
    s.full_name AS student_name,
    s.admission_no,
    cs.name || ' ' || cs.section AS class_label,
    emp.full_name AS class_teacher_name,
    ay.code AS academic_year,
    MIN(fi.due_date) AS earliest_due_date,
    SUM(fil.amount - fil.discount) - COALESCE(SUM(pa.amount), 0) AS pending_amount
FROM fee_invoices fi
JOIN enrolments e ON fi.enrolment_id = e.id
JOIN students s ON e.student_id = s.id
JOIN class_sections cs ON e.class_section_id = cs.id
LEFT JOIN employees emp ON cs.class_teacher_id = emp.id
JOIN academic_years ay ON fi.academic_year_id = ay.id
JOIN fee_invoice_lines fil ON fi.id = fil.invoice_id
LEFT JOIN payment_allocations pa ON fil.id = pa.invoice_line_id
WHERE fi.school_id = :school_id
    AND fi.status IN ('issued', 'partially_paid')
    AND fi.due_date < CURRENT_DATE
GROUP BY e.id, s.id, s.full_name, s.admission_no, cs.name, cs.section, emp.full_name, ay.code
HAVING (SUM(fil.amount - fil.discount) - COALESCE(SUM(pa.amount), 0)) > 0
ORDER BY pending_amount DESC;
```

Q28 What is an `idempotency\_key` on `fee\_payments`, and why is it critical?

`idempotency_key` is a unique `varchar(64)` string generated by the frontend/client for each payment submission.

- **Enforcement:** `UniqueConstraint("school_id", "idempotency_key")` on `fee_payments`.
- **Why it is critical:** If the fee counter clerk double-clicks the "Collect Fee" button, or if a network timeout causes a retry after the payment was already committed in the DB, PostgreSQL rejects the second insertion with a unique constraint violation. This physically prevents double-charging a parent or issuing two receipts for one payment.

Q29 How does payment reversal work, and what is a contra payment in the database?

When a payment is voided/bounced:

1. The original `fee_payments` record is **never deleted**.
2. A new `fee_payments` row is inserted with:
 - `amount = -original_payment.amount` (negative value),
 - `status = 'reversed'`,
 - `reverses_payment_id = original_payment.id`,
 - `reason = user_typed_audit_reason`.
3. Negative rows are inserted into `payment_allocations` for each line originally paid.
4. The related `fee_invoices.status` transitions from `'paid'` back to `'issued'` or `'partially_paid'`, and `settled_on` is cleared to `NULL`.
5. An audit row is logged in `audit_log`.

Q30 What are the columns and foreign keys of `fee\_heads`?

- **Columns:** `id`, `school_id` (FK `schools.id`), `name` (`varchar(60)`), `code` (`varchar(16)`), `type` (Enum: `tuition`, `transport`, `exam`, `facility`, `other`), `is_refundable` (`boolean`), `gl_code` (`varchar(20)`, General Ledger accounting code), `is_active` (`boolean`), `created_at`, `updated_at`.
- **Unique Constraint:** `UniqueConstraint("school_id", "code")` prevents duplicate head codes (e.g. `'TUIT'`) within the same school.

Q31 What are the foreign keys on `fee\_plan\_items`, and what constraint prevents duplicate heads on the same plan?

- `fee_plan_id` points to `fee_plans.id` (`onDelete="CASCADE"`).
- `fee_head_id` points to `fee_heads.id` (`onDelete="RESTRICT"`).
- `school_id` points to `schools.id` .
- **Constraint:** `UniqueConstraint("fee_plan_id", "fee_head_id")` . A single fee plan cannot list "Tuition Fee" twice; its amount and frequency must be specified in a single plan item.

Q32 How is frequency represented in `fee\_plan\_items`, and what values are allowed?

Stored in the `frequency` column as a `varchar(8)` string corresponding to the `FeeFrequency` enum:

- `'monthly'` : Billed every calendar month.
- `'term'` : Billed once per school term (e.g., Term 1, Term 2).
- `'annual'` : Billed once at the beginning of the academic year.
- `'one_time'` : Billed once upon initial enrolment (e.g., Admission Registration Fee).

Q33 How does the database record an approved student fee discount in `fee\_concessions`?

- **Columns:** `id` , `school_id` , `enrolment_id` (FK `enrolments.id`), `fee_head_id` (FK `fee_heads.id` , optional—if NULL, applies school-wide), `type` (Enum: `sibling` , `merit` , `staff_child` , `hardship` , `special`), `percent` (`numeric(5,2)`), `amount` (`numeric(10,2)`), `reason` (`text`), `status` (Enum: `pending` , `approved` , `rejected`), `requested_by` (FK `users.id`), `approved_by` (FK `users.id`), `decided_at` (`timestamptz`), `valid_from` , `valid_to` .
- Only concessions where `status = 'approved'` and current date falls between `valid_from` and `valid_to` are deducted during monthly invoice line generation.

Q34 What are the allowed values for `fee\_invoices.status`?

Managed by the `InvoiceStatus` enum:

1. `'issued'` : Invoice generated and sent to the parent; no payments applied yet.
2. `'partially_paid'` : One or more payments allocated, but remaining balance $\$ > 0\$$.
3. `'paid'` : Total allocations equal the full net amount of all invoice lines.
4. `'void'` : Cancelled by administration with an audited `void_reason` . No further payments can be allocated to this invoice.

Q35 What is the relationship between `fee\_invoices` and `fee\_invoice\_lines`?

A **One-to-Many** relationship:

- `fee_invoices.id` is the primary key.
- `fee_invoice_lines.invoice_id` is a non-nullable foreign key pointing to `fee_invoices.id` with `onDelete="CASCADE"` .
- When an invoice is deleted in test environments, its lines are removed automatically. In production, invoices are voided rather than deleted.

Q36 How does the system handle late fee calculations in the database?

Late fees are governed by policy settings in `settings` :

- `fees.late_fee.grace_days` (default `5` days after `due_date`),
- `fees.late_fee.initial` (default `₹300.00` flat charge once grace period expires),
- `fees.late_fee.per_day` (default `₹100.00` added per additional overdue day),
- `fees.late_fee.cap_percent` (ceiling cap, default `50%` of invoice amount).
- If overdue, an additional line item is dynamically appended to `fee_invoice_lines` referencing the `'LATE'` fee head code before payment calculation.

Q37 If an invoice has a gross amount of ₹5,000 and a concession discount of ₹500, what is stored in `fee\_invoice\_lines`?

- `fee_invoice_lines.amount` = 5000.00
- `fee_invoice_lines.discount` = 500.00
- Net payable = `amount - discount` = 4500.00 .
- Storing both `amount` and `discount` explicitly preserves financial visibility into the gross fee demand versus the scholarship awarded.

Q38 What happens if a parent pays ₹6,000 against an invoice of ₹4,500?

1. `fee_payments` records the full received amount: `amount = 6000.00` .
2. `payment_allocations` allocates 4500.00 across the lines of the current invoice, marking it 'paid' .
3. The remaining ₹1,500.00 is either allocated to the student's next unpaid invoice (FIFO) or left unallocated on `fee_payments` as an advance payment credit balance for future billing.

Q39 Why does `fee\_payments.enrolment\_id` point to `enrolments` rather than `students`?

Because fee payments are recognized within an **accounting session / academic year**.

- Linking to `enrolments` ensures that payments made in 2025-26 are audited strictly against 2025-26 enrolments, even if the student pays off previous years' arrears in a separate transaction.

Q40 What happens if an administrator tries to delete a `fee\_heads` row that already has invoice lines?

PostgreSQL rejects the operation with a Foreign Key Violation error:

```
psycopg.errors.ForeignKeyViolation: update or delete on table "fee_heads" violates foreign key constraint "fk_fee_invoice_lines_fee_head_id" on table "fee_invoice_lines".
```

This is because `fee_invoice_lines.fee_head_id` uses `onDelete="RESTRICT"` . To retire a fee head, the administrator must set `fee_heads.is_active = FALSE` .

Q41 Can an invoice be re-opened once marked `paid`?

Not directly. If a payment was attributed in error, the administrator executes a **contra payment reversal**. Once the contra allocation negates the previous allocation, the backend re-evaluates the invoice lines and automatically changes `status` from 'paid' back to 'issued' or 'partially_paid' .

Q42 What index on `fee\_invoices` ensures fast loading of the student fee ledger?

Index on `(school_id, enrolment_id, period_year, period_month)` . This enables immediate lookup of all monthly invoices for a given student within a session.

Q43 How does the system prevent generating two invoices for the same student for the same month?

By database query validation and unique indexing:

The generation service runs `SELECT id FROM fee_invoices WHERE enrolment_id = :e_id AND period_month = :m AND period_year = :y AND status ≠ 'void'` . If an invoice exists, the student is skipped.

Q44 What columns exist on `payment\_allocations`?

- `id` (bigint, PK)
- `school_id` (bigint, FK `schools.id`)
- `payment_id` (bigint, FK `fee_payments.id` with `onDelete="CASCADE"`)
- `invoice_line_id` (bigint, FK `fee_invoice_lines.id` with `onDelete="RESTRICT"`)
- `amount` (numeric(10,2))
- `created_at` (timestamptz)
- `updated_at` (timestamptz)

Q45 How is transport fee integrated into monthly student invoices?

1. `transport_assignments` tracks which active bus stop (`route_stops`) the child is assigned to.
2. `route_stops.fee_slab_id` points to `transport_fee_slabs` (`monthly_amount`).
3. During invoice generation, the backend checks for an active `transport_assignments` record for the enrolment.
4. If found, it automatically appends a `fee_invoice_lines` entry for `fee_head = 'Transport Fee'` with the slab's `monthly_amount`.

Q46 Can a fee payment be made in multiple payment modes (e.g. part Cash, part UPI)?

Yes, by inserting **two separate rows** in `fee_payments`:

- Row 1: `amount = 2000`, `method = 'cash'`, `receipt_no = RCP-001`
- Row 2: `amount = 3000`, `method = 'upi'`, `receipt_no = RCP-002`, `instrument_ref = 'UPI/UTR/12345'`

Each payment independently allocates to the invoice lines via `payment_allocations`.

Q47 What prevents a fee collector from altering an invoice amount to give an unauthorized discount?

1. **API Layer:** There is no `PATCH /admin/fees/invoices/{id}` endpoint that accepts amount modifications.
2. **Concession Layer:** Concessions must be pre-approved in `fee_concessions` with `approved_by` referencing a user holding the `fees.concession.approve` permission.
3. **Database Audit:** Any voiding of invoices writes the actor's ID and mandatory reason to `audit_log`.

Q48 How is the total fees collected vs fees remaining computed for the Executive Dashboard?

`backend/app/services/stats.py::totals`:

- **Fees Collected:** `SELECT COALESCE(SUM(amount), 0) FROM fee_payments WHERE school_id = :sid AND status = 'cleared'`
- **Fees Invoiced:** `SELECT COALESCE(SUM(amount - discount), 0) FROM fee_invoice_lines WHERE school_id = :sid AND invoice_id IN (SELECT id FROM fee_invoices WHERE school_id = :sid AND status ≠ 'void')`
- **Fees Remaining:** `Fees Invoiced - Fees Collected`.

Q49 What is the `fee\_periods` table in the database?

`fee_periods` represents formal monthly/quarterly accounting periods.

- It stores `period_month`, `period_year`, `is_closed`, `closed_at`, `closed_by`.
- When an accounting month is marked "Closed", the backend rejects any back-dated invoice creation or void actions for that closed period.

Q50 How does the system ensure that when an invoice is voided, previous payments are not orphaned?

The backend verifies that no active `payment_allocations` exist for any line of that invoice:

- If allocations exist, the backend refuses the void request with an error: `*"Cannot void an invoice with active payments. Reverse payments first."*`
- Only unallocated or fully reversed invoices can be marked `'void'`.

MODULE 4: ACADEMICS, TIMETABLE & HOMEWORK**Q51 What tables represent the classroom hierarchy in the database?**

- **class_sections**: A physical class section (e.g. "Class 10-A").
- Columns: `id`, `school_id`, `name` (e.g. '10'), `section` (e.g. 'A'), `class_label` (e.g. '10-A'), `class_teacher_id` (FK `employees.id`), `room_number`, `capacity`.
- **subjects**: Subjects taught in the school (e.g. "Mathematics", "Science").
- **class_subject_teacher**: Junction assigning a subject teacher to a specific class section.

Q52 How does `class\_subject\_teacher` allocate teachers to subjects?

`class_subject_teacher` contains:

- `class_section_id` (FK `class_sections.id`)
- `subject_id` (FK `subjects.id`)
- `teacher_id` (FK `employees.id`)
- `periods_per_week` (Integer, e.g. 6)
- `is_primary` (Boolean)
- **Constraint:** `UniqueConstraint("class_section_id", "subject_id", "teacher_id")`.

Q53 What tables power the school timetable, and how are weekly slots structured?

Two tables:

1. **`school_periods`**: Defines the bell schedule (e.g., Period 1: 08:00–08:45, Period 2: 08:45–09:30, Break: 10:15–10:35).
 - Columns: `id`, `school_id`, `period_number`, `name`, `start_time`, `end_time`, `is_break`.
2. **`timetable_slots`**: The actual weekly timetable cell.
 - Columns: `id`, `school_id`, `class_section_id`, `day_of_week` (Enum: `monday – saturday`), `period_id` (FK `school_periods.id`), `subject_id` (FK `subjects.id`), `teacher_id` (FK `employees.id`), `room_number`.

Q54 How does the database prevent a teacher from being scheduled in two different classrooms at the same time?

Enforced by a composite database unique constraint on `timetable_slots`:

`UniqueConstraint("school_id", "day_of_week", "period_id", "teacher_id")`.

If an administrator attempts to assign Teacher A to Class 9-A and Class 10-B during Monday Period 3, PostgreSQL rejects the second insertion with a unique key conflict.

Q55 How does the database prevent two different classes from being scheduled in the same room at the same time?

Enforced by:

`UniqueConstraint("school_id", "day_of_week", "period_id", "room_number")` on `timetable_slots` (where `room_number` IS NOT NULL).

Q56 What table stores homework assignments given by teachers?

homework:

- **Columns:** `id`, `school_id`, `class_section_id` (FK), `subject_id` (FK), `teacher_id` (FK `employees.id`), `title`, `description`, `assigned_date`, `due_date`, `max_marks`, `attachments` (JSON array of document URLs).

Q57 How does `homework\_submissions` track student homework completion?

- **Columns:** `id`, `school_id`, `homework_id` (FK `homework.id`), `enrolment_id` (FK `enrolments.id`), `submitted_at`, `content`, `attachments`, `status` (Enum: `pending`, `submitted`, `evaluated`, `late`), `marks_obtained`, `feedback`, `evaluated_by` (FK `employees.id`).
- **Constraint:** `UniqueConstraint("homework_id", "enrolment_id")` ensures a student can submit homework only once per assignment.

Q58 What is the `holidays` table, and what is its role in academic scheduling?

- **Columns:** `id`, `school_id`, `academic_year_id` (FK), `name`, `start_date`, `end_date`, `holiday_type` (`gazetted`, `restricted`, `vacation`), `description`.
- When calculating working days for student attendance percentages, days falling within `holidays` are excluded from the required attendance denominator.

Q59 Why does `homework\_submissions` link to `enrolment\_id` instead of `student\_id`?

Because homework is assigned to a specific **class section within an academic year**. Linking to `enrolment_id` guarantees that submissions are cataloged against the child's specific section record for that session.

Q60 How does the backend enforce the maximum teaching load policy for a teacher?

`settings` defines `timetable.max_periods_per_week` (default `30`).

- Before inserting into `timetable_slots`, the service counts existing slots:

```
SELECT count(*) FROM timetable_slots WHERE teacher_id = :t_id.
```

- If the count exceeds the threshold, the operation is blocked unless authorized with a leadership override.

Q61 What happens to `timetable\_slots` if a subject is deleted?

`timetable_slots.subject_id` is set to `ondelete="RESTRICT"`. You cannot delete a subject from the catalog if classes have scheduled timetable periods for it.

Q62 What is the relationship between `class\_sections` and `employees`?

`class_sections.class_teacher_id` is a nullable foreign key pointing to `employees.id` (`ondelete="SET NULL"`). An employee serves as the primary Class Teacher responsible for daily attendance roll-call and report card remarks.

MODULE 5: STUDENT ATTENDANCE**Q63 What table records daily student attendance, and what are its columns?**

attendance :

- `id` (bigint, PK)
- `school_id` (bigint, FK `schools.id`)
- `enrolment_id` (bigint, FK `enrolments.id`)
- `date` (date without time zone)
- `status` (Enum: `present`, `absent`, `late`, `half_day`, `excused`)
- `remarks` (`varchar(160)`, optional explanation)
- `recorded_by` (bigint, FK `users.id`)
- `created_at`, `updated_at` (`timestampz`)

Q64 What constraint prevents marking duplicate attendance for a student on the same day?

`UniqueConstraint("school_id", "enrolment_id", "date")` on the `attendance` table.

If a teacher submits roll call twice for the same morning, the database rejects the duplicate row, allowing the backend to perform an `upsert` (update existing status) instead of creating duplicate records.

Q65 Why does `attendance` link to `enrolment\_id` instead of `student\_id`?

Per Design Rule 2: Attendance is a **year-scoped classroom fact**. A student attends Class 10-A in session 2025-26 as an **enrolment**. If it pointed at `student_id`, attendance from Class 9 would conflict or bleed into Class 10 queries for the same calendar date.

Q66 How is the CBSE 75% short attendance threshold calculated by the backend?

1. `settings` stores `attendance.shortage_threshold = 75`.
2. The query evaluates:

Formula: $\text{Attendance \%} = \frac{\text{COUNT(attendance WHERE status = 'present')}}{\text{COUNT(total working days)}} \times 100$

3. Any student whose computed percentage is $\leq 75\%$ is flagged on the `/attendance` shortage list and debars the student from automatic admit card generation.

Q67 What are the allowed values for `attendance.status`?

Managed by the `AttendanceStatus` enum:

- `'present'` : Student present in class.
- `'absent'` : Unauthorized absence.
- `'late'` : Arrived after the morning bell.
- `'half_day'` : Left early or arrived after lunch.
- `'excused'` : Authorized absence with approved leave letter.

Q68 What index ensures fast loading of the class roll-call screen?

Composite index on `(school_id, date, enrolment_id)`. This allows the attendance sheet for 40 students in a section to load in <5 milliseconds.

Q69 What is the `student\_leave\_requests` table, and how does it relate to `attendance`?

`student_leave_requests` stores parent leave applications:

- Columns: `id`, `school_id`, `enrolment_id`, `start_date`, `end_date`, `reason`, `status` (`pending`, `approved`, `rejected`), `approved_by`.
- When a leave request is marked `'approved'`, the attendance marking engine defaults that student's status to `'excused'` on the roll-call register for those dates.

Q70 Does the attendance table support periods-wise attendance or daily attendance?

The active operational database implements **daily attendance** (`date` column). Period-wise tracking is modeled in architecture blueprints but daily morning roll call is the operational system of record.

MODULE 6: EXAMINATIONS, MARKS & CBSE GRADING**Q71 What tables store exam definitions, schedules, and student marks?**

- **exams** : The exam event (e.g., "Term 1 Midterm Exam 2025-26", `academic_year_id`, `status`).
- **exam_schedule** : Timetable for each subject paper (Exam ID, Subject ID, Class Section ID, Exam Date, Max Marks, Passing Marks).
- **marks** : The actual score obtained by a student for a specific paper.

Q72 What columns exist on the `marks` table?

- `id` (bigint, PK)
- `school_id` (bigint, FK `schools.id`)
- `exam_schedule_id` (bigint, FK `exam_schedule.id`)
- `enrolment_id` (bigint, FK `enrolments.id`)
- `marks_obtained` (numeric(5,2))
- `is_absent` (boolean, defaults to FALSE)
- `remarks` (varchar(120))
- `entered_by` (bigint, FK `users.id`)
- `is_locked` (boolean, defaults to FALSE)
- **Constraint:** `UniqueConstraint("exam_schedule_id", "enrolment_id")` ensures one mark record per paper per student.

Q73 What is the "Paper Lock" feature, and how does the database enforce it?

Once marks entry is complete, the academic coordinator locks the paper to prevent grade tampering:

- `exam_schedule.is_locked` is set to `TRUE`.
- Any subsequent `POST` or `PATCH` to `/admin/exams/{id}/marks` queries `exam_schedule.is_locked`. If `TRUE`, the backend raises `403 Forbidden: "Marks entry is locked for this paper"`.
- Overriding a locked paper requires the `exam.marks.override` permission, updates `marks.is_locked`, and writes a mandatory justification to `audit_log`.

Q74 How are CBSE assessment schemes and components modeled in the database?

Two tables:

1. **assessment_schemes**: (e.g. "CBSE Secondary Assessment 2025-26", `theory_weightage` = 80, `practical_weightage` = 20).
 2. **scheme_components**: Individual exam parts (e.g., "Periodic Test (10 marks)", "Notebook Submission (5 marks)", "Subject Enrichment (5 marks)", "Annual Board Exam (80 marks)").
- Columns: `scheme_id`, `name`, `max_marks`, `weightage_percent`, `sequence`.

Q75 How does the database calculate letter grades using `grading\_scales` and `grade\_bands`?

- **grading_scales**: e.g., "CBSE 8-Point Secondary Scale".
- **grade_bands**: Defines the score boundaries:
- `grade_point` = 10, `letter_grade` = 'A1', `min_percentage` = 91.00, `max_percentage` = 100.00
- `grade_point` = 9, `letter_grade` = 'A2', `min_percentage` = 81.00, `max_percentage` = 90.99
- ... down to `letter_grade` = 'E' (Fail).
- When report cards are computed, the backend calculates the percentage:

Formula: $\text{Percentage} = \frac{\sum \text{marks_obtained}}{\sum \text{max_marks}} \times 100$

Then matches against `grade_bands` WHERE `:percent BETWEEN min_percentage AND max_percentage`.

Q76 How does the database implement policy \$0.6b: "Withholding report cards for unpaid fee dues"?

- `settings` stores `exams.withhold_results_for_dues` = `True`.
 - When a student or parent requests report cards via `/parent/results` or `/student/results`:
1. The backend runs the fee balance query for that `enrolment_id`.
 2. If $\text{Pending Balance} > 0$, the endpoint does not return the marks dossier.
 3. Instead, it returns `status: "withheld"` with message: `"Report card withheld due to outstanding fees of ₹X,XXX.00. Contact accounts office."`

Q77 What happens in `marks` if a student was absent for an exam?

- `marks.is_absent` is set to `TRUE`.
- `marks.marks_obtained` is stored as `0.00`.
- The report card generator renders `'AB'` on the printed grade sheet and excludes the paper from aggregate total calculations where permissible under CBSE bylaws.

Q78 Can an exam be scheduled on a Sunday or official school holiday?

The backend validates `exam_schedule.exam_date` against `holidays`:

- If `exam_date` matches a row in `holidays` for that `academic_year_id`, the API returns a warning or blocks the schedule without administrator override.

Q79 What is the relationship between `exams` and `academic\_years`?

`exams.academic_year_id` is a foreign key pointing to `academic_years.id` (`onDelete="RESTRICT"`). All exam series, schedules, and marks belong to a specific academic session.

Q80 What is the `report\_card\_publications` table?

Tracks official report card release batches:

- Columns: `id`, `school_id`, `exam_id`, `class_section_id`, `published_at`, `published_by`, `withheld_count`, `released_count`.

MODULE 7: TRANSPORT & FLEET MANAGEMENT**Q81 What 5 tables represent the Transport module in the database?**

- vehicles**: Physical buses/vans (registration no, seating capacity, insurance expiry, fitness certificate).
- routes**: Travel circuits (route code, route name, driver, attendant, vehicle).
- route_stops**: Halts along a route with scheduled morning pickup and evening drop times.
- transport_fee_slabs**: Pricing bands based on distance (e.g., Slab 1 [0-5km] = ₹1,200/mo).
- transport_assignments**: Student passenger allocations linking enrolments to bus stops.

Q82 How does the database prevent over-allocating students to a bus beyond its legal seating capacity?

- `vehicles.capacity` stores the legal seating capacity from the RTO registration papers.
- `routes.vehicle_id` links the bus to the route.
- Before assigning a student in `transport_assignments`, the backend runs:

```
SELECT count(*) FROM transport_assignments ta
JOIN route_stops rs ON ta.route_stop_id = rs.id
WHERE rs.route_id = :route_id AND ta.status = 'active';
```

- If active count $\geq$ `vehicles.capacity`, the database transaction aborts:

400 Bad Request: "Vehicle capacity of 42 exceeded. Cannot allocate student."

Q83 What columns exist on `transport\_assignments`?

- `id` (bigint, PK)
- `school_id` (bigint, FK `schools.id`)
- `enrolment_id` (bigint, FK `enrolments.id`)
- `route_stop_id` (bigint, FK `route_stops.id`)
- `direction` (Enum: `two_way`, `pickup_only`, `drop_only`)
- `start_date` (date)
- `end_date` (date, nullable)
- `status` (Enum: `active`, `suspended`, `cancelled`)
- `end_reason` (text, audited reason for ending transport)

Q84 What is the relationship between `routes`, `employees`, and `vehicles`?

- `routes.vehicle_id` points to `vehicles.id` (Many routes to one vehicle, though typically 1:1 active).
- `routes.driver_id` points to `employees.id` (where employee is support staff/driver).
- `routes.attendant_id` points to `employees.id` (bus conductor/attendant).

Q85 How does `route\_stops` connect to `transport\_fee\_slabs`?

Each halt row in `route_stops` holds a foreign key `fee_slab_id` pointing to `transport_fee_slabs.id`.

When a child is assigned to that stop, the invoicing system looks up `route_stops.fee_slab_id` → `transport_fee_slabs.monthly_amount` to bill the exact distance fee.

Q86 What unique constraint ensures bus stops are ordered correctly on a route?

```
UniqueConstraint("route_id", "sequence") on route_stops.
```

Stop 1, Stop 2, Stop 3 cannot share sequence numbers, ensuring the morning bus timetable is strictly ordered.

MODULE 8: STOCK & INVENTORY MANAGEMENT**Q87 What tables manage stock and supplies in Sunrise ERP?**

1. **stock_items** : The catalog of consumable supplies and science lab materials (Item Name, Code, Category, Unit, Quantity on Hand, Minimum Reorder Threshold).
2. **stock_requests** : Material requisition tickets submitted by teachers or staff for restocking or issuance.

Q88 What columns exist on `stock\_items`, and how are low-stock alerts detected?

- **Columns:** `id`, `school_id`, `item_code` (`varchar(20)`), `name` (`varchar(100)`), `category` (Enum: `lab_supplies`, `stationery`, `sports`, `medical`, `maintenance`), `unit` (e.g. `'boxes'`, `'litres'`, `'pieces'`), `quantity_on_hand` (`integer`), `minimum_quantity` (`integer`), `unit_price` (`numeric(10,2)`), `location` (e.g. `'Chemistry Lab Cupboard B'`).

- **Low-stock alert query:**

```
SELECT * FROM stock_items WHERE school_id = :sid AND quantity_on_hand ≤ minimum_quantity;
```

Any item where current inventory drops below the safety buffer triggers a red status badge on the admin inventory screen.

Q89 What happens in the database when a teacher flags diminishing stock from the mobile app?

When `POST /teacher/stock/flag` is sent:

1. Validates that `stock_items.id` exists and teacher is authenticated.
2. Inserts a row into **stock_requests** :
 - `item_id = item.id`,
 - `requested_by = user.id`,
 - `request_type = 'depletion_flag'`,
 - `quantity = flagged_quantity`,
 - `status = 'pending'`,
 - `remarks = teacher_notes`.
3. The ERP admin immediately sees the flag on the `/inventory` screen with the teacher's name and timestamp.

Q90 What database updates occur when the admin approves a stock request?

Inside an atomic transaction:

1. `stock_requests.status` is updated to `'approved'`.
2. `stock_requests.decided_by` is set to `admin.id` and `decided_at = now()`.
3. If it is a restocking/purchase approval, `stock_items.quantity_on_hand += stock_requests.quantity`.
4. If it is an issuance to a classroom, `stock_items.quantity_on_hand -= stock_requests.quantity`.
5. An audit row is added to `audit_log`.

Q91 What unique constraint exists on `stock\_items`?

```
UniqueConstraint("school_id", "item_code").
```

No two items can share the same SKU/code within the same school (e.g., `'CHEM-001'` is unique).

MODULE 9: GRIEVANCES & FEEDBACK

Q92 What two tables implement the Grievance and Helpdesk system?

1. **grievances** : The primary ticket header.
2. **grievance_replies** : The chronological conversational thread of replies between parents, teachers, and administrators.

Q93 What columns exist on `grievances`?

- **id** (bigint, PK)
- **school_id** (bigint, FK **schools.id**)
- **ticket_no** (varchar(24) , unique per school, e.g. 'GRV-2025-0004')
- **submitter_id** (bigint, FK **users.id**)
- **submitter_role** (Enum: **parent** , **teacher** , **student** , **staff**)
- **student_id** (bigint, nullable FK **students.id** —filled when a parent files a complaint about their child)
- **category** (Enum: **academics** , **transport** , **fees** , **facilities** , **safety** , **general**)
- **priority** (Enum: **low** , **medium** , **high** , **urgent**)
- **status** (Enum: **open** , **in_progress** , **resolved** , **closed**)
- **title** (varchar(160))
- **description** (text)
- **assigned_to** (bigint, nullable FK **employees.id** —staff member tasked with solving the issue)
- **resolution_notes** (text , explanation recorded when closing ticket)
- **resolved_at** (timestampz)

Q94 How does `grievance\_replies` support multi-party conversation?

- **id** (bigint, PK)
- **grievance_id** (bigint, FK **grievances.id** with **onDelete="CASCADE"**)
- **user_id** (bigint, FK **users.id**)
- **message** (text)
- **attachments** (JSON array of image/document URLs)
- **is_internal** (Boolean: if TRUE, only visible to administrators as private internal notes)
- **created_at** (timestampz)

Q95 What happens in the database when an admin assigns a grievance to a teacher?

1. **PATCH /admin/grievances/{id}** updates **grievances.assigned_to = employee_id** .
2. **grievances.status** transitions from **'open'** to **'in_progress'** .
3. When the teacher logs in to their mobile app, the query **SELECT * FROM grievances WHERE assigned_to = :my_employee_id** displays the assigned ticket under their "Assigned to Me" tab.

MODULE 10: HUMAN RESOURCES, COMMUNICATION & SYSTEM AUDIT

Q96 What tables store employee and department records?

- **departments** : Academic and administrative departments (e.g., Mathematics Dept, Science Dept, Transport Dept, Accounts).
- Columns: **id** , **school_id** , **name** , **code** , **head_teacher_id** (FK **employees.id**).
- **employees** : Staff profiles.
- Columns: **id** , **school_id** , **user_id** (FK **users.id** , nullable for non-login staff like cleaners), **department_id** (FK **departments.id**), **employee_code** (e.g. 'TCH001'), **full_name** , **email** , **phone** , **employee_type** (**teaching** , **administrative** , **support**), **designation** , **status** (**active** , **exited**), **joining_date** , **qualification** .

Q97 What is the relationship between `users` and `employees`?

A **One-to-One / Many-to-One optional foreign key**: `employees.user_id → users.id`.

- Teaching and administrative staff have a `user_id` to log in to the web ERP or mobile app.
- Support staff (cleaners, ground staff) exist in `employees` for payroll and contact directories, but their `user_id` is `NULL` because they do not have system logins.

Q98 What tables power school notices and announcements?

- notices**: Circulars published by school management.
- Columns: `id`, `school_id`, `title`, `content`, `audience` (Enum: `all`, `parents`, `teachers`, `students`), `posted_by` (FK `users.id`), `is_published`, `expires_on`, `attachments`.
- message_templates**: Pre-defined SMS/Email formats for fee reminders, attendance warnings, and holiday greetings.

Q99 What are the columns and performance indexing of `audit\_log`?

- Columns**: `id`, `school_id`, `user_id`, `action`, `entity_type`, `entity_id`, `reason`, `before_state` (JSON), `after_state` (JSON), `ip_address`, `created_at`.
- Indexes**:
 - `(school_id, created_at DESC)` for instant chronological streaming on the admin security monitor.
 - `(school_id, entity_type, entity_id)` to view the full audit history of a specific invoice or student record.

Q100 What is stored in `documents` and `document\_types`?

- document_types**: Standard certificate categories required by school policy (e.g., Birth Certificate, Transfer Certificate, Aadhaar Card, Previous Marksheet).
- documents**: Uploaded file metadata:
 - Columns: `id`, `school_id`, `document_type_id` (FK), `entity_type` (`'student'`, `'employee'`), `entity_id`, `file_name`, `file_url`, `file_size`, `mime_type`, `verified_by` (FK `users.id`), `verified_at`.
 - The actual binary file is stored in local storage (`./var/documents/`) or cloud S3; the database stores only metadata and file paths.

★ The Most Important 20 Questions to Prepare First

Master these 20 high-yield questions first to demonstrate thorough understanding of multi-tenancy, financial immutability, data flows, and constraints.